

3-Mark Questions & Answers - Data Structures Module 1

Topic 1: Concept of ADT

Q1. Explain the concept of ADT with a suitable example.

An Abstract Data Type (ADT) is a high-level specification of a set of data items and the operations performed on them, completely independent of how the data is stored in memory or how operations are implemented.

* **Example: List ADT**

* **Da

Q2. What are the advantages of using ADTs in programming?

1. Modularity & Reusability: Software is broken into independent modules. An ADT implementation can be modified or optimized without altering client code.

2. Infor
details

Q3. Explain the relationship between ADT and encapsulation.

* **Encapsulation** is the object-oriented programming mechanism of wrapping data and methods into a single unit (class/structure) and restricting direct access to private data.

* **P
provide
opaque

Topic 2: Types of Data Structures

Q4. Classify data structures with a neat diagram.

Data structures are categorized into Primitive and Non-Primitive, with Non-Primitive further split into Linear and Non-Linear structures:

Data Structures

|

Q5. Explain primitive vs non-primitive data structures with examples.

| Property | Primitive Data Structures | Non-Primitive Data Structures |

| :--- | :-

Q6. Compare linear and non-linear data structures (any three points).

| Feature | Linear Data Structures | Non-Linear Data Structures |

| :--- | :-

Topic 3: Operations on Data Structures

Q7. Explain insertion and deletion operations with examples.

* **Insertion Operation:** The process of adding a new data element into an existing data structure.

* **Exa
capacity

Q8. Explain sorting and merging operations on data structures.

* **Sorting Operation:** Arranging elements of a data structure in a specific numerical or alphabetical order (Ascending or Descending).

* **Exa

Q9. Differentiate between traversal and searching operations.

| Feature | Traversal Operation | Searching Operation |

| :--- | :-

Topic 4: Arrays & Multi-Dimensional Arrays

Q10. Explain row-major and column-major order representation of a 2-D array.

A 2-D array is logically a matrix but stored in 1-D contiguous physical memory:

1. Row-
2, etc. (

Q11. Derive the address calculation formula for an element in a 2-D array (row-major).

For a 2-D array $A[m][n]$ stored in Row-Major Order with Base Address B and element size w :

* To re

Equation:

Address

(Assuming $LBR = LBC = 0$, $Address = B + w \times (i \times n + j)$).

Q12. Write a short note on sparse matrices.

A Sparse Matrix is a matrix in which the majority of elements are zero.

* **Pr
significa

Topic 5: String Manipulation

Q13. Explain any three string manipulation functions with examples.

1. `strlen(s)`: Calculates length excluding null character.

* *Exa

Q14. Write a note on pattern matching in strings.

Pattern Matching is the process of checking whether a specific substring (called Pattern P of length m) exists inside a larger text string (called Text T of length n).

* **Na

Q15. Explain the concept of string representation as an array of characters.

In languages like C, strings are represented as 1-D arrays of `char` primitive elements.

* **Sto

Topic 6: Self-Referential Structures with Pointers

Q16. Explain self-referential structures with a suitable example and diagram.

A Self-Referential Structure contains a pointer variable pointing to another structure of the same type.

struct N

Q17. Explain how self-referential structures are used to implement linked lists.

Dynamic linked lists cannot use static contiguous arrays. Instead:

1. Each

Q18. Differentiate between arrays and self-referential (pointer-based) structures.

| Feature | Arrays | Self-Referential Structures |

| :--- | :-